03/27 PRESENTATION

Hardware Meeting

JC Vaught
Mechanical Engineering
M.S. Student

OUTLINE

1. Real-Time Radar to Camera Projection
2. Furuno Radar - Rust Reimplementation
3. FLIR M364C Driver Rewrite
4. Aligned Thermal/RGB Data
5. Automated IMU Calibration Testing
6. VLM-Controlled Segmentation
7. Compute Acquisition
8. Future Work

REAL-TIME RADAR TO CAMERA PROJECTION

We can just fix it in post.

THE PROBLEM & PRIOR STATE

What Existed

- Ty's system: C++/Python hybrid
- Post-processing only (Python/MATLAB)
- ROS2 middleware for inter-process comms
- ROS2 overhead: 8–15ms image pub/sub, 30–50MB RAM per node, 2–5s startup

What Was Needed

- Real-time overlay at 30 fps ($<$33ms budget)
- Sensor fusion: Lucid camera + SBG IMU/GPS + Simrad radar
- Web-accessible GUI for remote monitoring
- No ROS2 dependency

SYSTEM ARCHITECTURE

Built entirely in Rust over a 4-day sprint. Components:

Drivers

- **Lucid camera** — Arena SDK wrapped via Rust FFI (bindgen)
- **SBG Ellipse IMU+GPS** — sbgECom protocol wrapped via FFI
- **Simrad radar** — fully written from scratch in Rust; reverse-engineered multicast UDP protocol (no official SDK)

Application Layer

- **File writer** — parallel multi-format raw writes
- **Network manager** — auto-detect ports, configure on boot
- **Orchestrator** — central controller
- **Projection engine** — custom CUDA kernels
- **Web GUI** — WebSocket server to browser

PERFORMANCE

CUDA Pipeline Stages

- Raw Bayer $\rightarrow$ RGB demosaic: $\sim$0.2ms
- RGB $\rightarrow$ JPEG encode (nvJPEG): $\sim$1.0ms
- Downscale to streaming resolution: $\sim$0.1ms
- Radar $\rightarrow$ camera coordinate projection: $<$0.1ms
- WebSocket delivery to browser: $<$1ms

Metric	ROS2 (FastDDS)	Custom Rust
Image transport (5MP)	8–15ms	$<$0.2ms
End-to-end pipeline	30–80ms	**$<$15ms (often $<$10ms)**
Memory per node	30–50MB	2–5MB

STATUS & NEXT STEPS

Working

- Full pipeline functional end-to-end (camera $\rightarrow$ browser with projection)
- All three sensor drivers operational
- Web GUI streaming with real-time radar overlay

Remaining

- Bug fixes from the 4-day sprint
- Handoff to Ty for continued development
- Integration with unified coordinate system (see Future Work)

FURUNO RADAR - RUST REIMPLEMENTATION

No SDK? No problem.

FURUNO RADAR - WHY REWRITE?

The Problem

- Furuno's official SDK (`libNAVNETDLL`) is a closed-source, Windows-only binary
- No documentation of the wire protocol
- Cannot run on Linux field computers
- No web interface, no built-in recording

The Solution

- Reverse-engineered the entire protocol from packet captures
- Reimplemented from scratch in Rust — zero proprietary dependencies
- Cross-platform (Linux, macOS, Windows), single binary
- Browser-based dashboard for remote control and monitoring

ARCHITECTURE & FEATURES

Protocol Layer

- Authentication handshake
- 90+ register commands
- 4 echo encoding formats
- Doppler processing
- Auto-discovery

Connection Manager

- 5-stage state machine
- Auto-reconnection
- Keepalive management
- Network auto-config

Web Dashboard

- Real-time PPI display
- Full settings control (range, gain, clutter, STC)
- One-click CSV recording with metadata
- Automated parameter sweeps

FLIR M364C DRIVER REWRITE

Threads are fun. . .

WHAT BROKE & WHY

The Bug

- Original C++ driver used multiple writer threads but a *single shared mutex*
- At low FPS / single format: contention was masked
- At higher FPS + multi-format writes (thermal + visible + radiometric): writers serialize on the lock
- Each writer held the mutex during disk I/O — blocking all others

Consequence

- Dropped frames from growing queues
- Memory leak — queues grew unbounded when writers couldn't keep up
- System became unusable at target frame rates

WHAT WAS FIXED

1. **Per-writer threading** — eliminated shared mutex during I/O; each writer has its own queue
2. **UUIDv7 image referencing** — each frame gets a unique, time-sortable UUID (RFC 9562); stored in EXIF `ImageUniqueID` and XMP `dc:Identifier`
3. **Metadata in EXIF/XMP** — moved metadata out of filenames and into the JPEG image data itself (emissivity, distance, palette, camera params)
4. **Master metadata manifest** — updated the central manifest file to index by UUID with full sensor parameters

Both thermal and visible frames from the same capture share the same UUID for pairing.

ALIGNED THERMAL/RGB DATA

Hard to align when you’re not on the same wavelength

TWO EXPERIMENTS

Experiment 1: Physical Offset

- Measured center-to-center lens separation: 6.5cm
- Static scene: moved entire rig by exactly the lens offset
- Effectively co-located both lenses at the same position across two captures
- Result: perfectly aligned thermal/RGB pairs

Experiment 2: Controlled Parallax

- At 6.5cm baseline, parallax is sub-pixel beyond $\sim$200ft
- Setup: top of parking garage, objects at 200–500ft
- Entire frame is effectively parallax-free (aligned)
- Then moved camera sideways by several feet $\rightarrow$ synthetic misalignment
- Result: controlled pairs with known alignment/misalignment

AUTOMATED IMU CALIBRATION TESTING

Stability is hard to come by these days

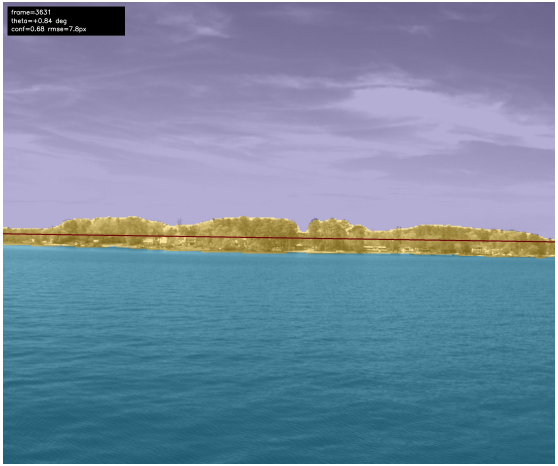
HORIZON-BASED STABILIZATION EVAL.

Problem: Quantify how un-level an IMU-stabilized camera is after "stabilization."

Method

1. Semantic segmentation $\rightarrow$ land/water/sky masks
2. Water-sky boundary $\rightarrow$ RANSAC line fit
3. Horizon angle: $\theta = \arctan(\text{slope})$
4. θ deviation over time $\rightarrow$ RMS = quality

Method	RMS	Latency
Exp. MA	1.5°	50–200ms



VLM-CONTROLLED SEGMENTATION

Maybe the machines can take my job too

THE PROBLEM WITH SAM

SAM 2 (Meta, 2024) is state-of-the-art for promptable segmentation, but:

- When you click on different parts of the *same object*, SAM refuses to grow the mask
- Instead it produces separate masks or ignores new clicks entirely
- **Root causes:**
  - Decoder biased toward compact, semantically “tight” segments
  - Training data (SA-1B): each mask = one coherent visual region
  - Point prompts on distant regions confuse the decoder’s attention
- SAM 2.1 improves video tracking but does not fix this
- SAM 3 (Nov 2025) adds Promptable Concept Segmentation (text/image $\rightarrow$ all instances) but the per-object multi-click limitation persists

This is a known community pain point with no clean solution in the literature.

SUB-MASKS + VLM CONTROL

Novel Contribution: Sub-Mask Union

1. Click on each visually distinct part of the object separately
2. Get individual sub-masks for each part from SAM
3. Union all sub-masks $\rightarrow$ complete object mask
4. Morphological closing to fill gaps

Practiced informally in annotation tools but *never formalized or evaluated systematically*.

VLM as Controller: Qwen2-VL-8B (7.6B params) reasons about where to click, when to zoom, when to stop.

Current Results

- Beats SOTA models on edge cases (where SAM's single-click fails)
- Does not beat SOTA on average — using *untrained* models only
- **Next step:** RL training (GRPO) to learn tool-use policy (zoom, super-resolution, sub-mask creation)

COMPUTE ACQUISITION

Compute? Just do it by hand

WHY I NEED COMPUTE

Current Hardware: 8× RTX 6000 Ada (48GB VRAM each)

RL Training Requirements for Qwen2-VL-8B

- GRPO + LoRA: 2–4× A100 80GB, 50–200 GPU-hours
- Full PPO: 4–8× A100 80GB, 100–500 GPU-hours
- Multiple experiment iterations needed → cumulative demand is high

The Gap

- RTX 6000 Ada (48GB) can fit single-model inference and LoRA fine-tuning
- PPO requires ∼48GB just for weights (policy + value + reference) — no room for activations
- GRPO needs ∼32GB (policy + reference) + activations → tight fit, limits batch size
- I need A100 80GB or H100 class hardware for full-scale RL training at reasonable throughput

ALLOCATION LANDSCAPE

Tier	Program	Amount	Effort	Timeline
1	ACCESS Explore	400K credits (∼400 A100-hrs)	Abstract	Days
1	NAIRR Startup	Up to 2K GPU-hrs	2-page app	2 weeks
2	ACCESS Discover	1.5M credits	1-page	Rolling
2	ACCESS Accelerate	3M credits	3-page	Panel
2	NAIRR Full	12-mo, A100/H100	3-page PDF	Periodic
3	ACCESS Maximize	No cap	10-page	Jun–Jul 2026
3	DOE ALCC	Frontier, Aurora	5–10 page	∼Nov 2026
3	DOE INCITE	500K–1M+ node-hrs	Full proposal	∼Apr 2026
4	Google Cloud	$1K–$5K	Online form	Rolling
4	AWS Research	Up to $5K (students)	Online form	Rolling
4	Azure Students	$100/yr	No CC needed	Rolling

FUTURE WORK

TODO: finish TODO list

UNIFIED COORDINATE SYSTEM

Working on a unified coordinate frame across multiple cameras and radar. The Florida / Jacob Minsky team also needs this for their work.

- Feeds into cue-to-slew, radar→camera projection, and camera→radar verification
- Enables single variable-zoom camera replacing the current dual-camera setup
- Feature-point-based manual calibration replacing the existing fully-human-based tuning (Dr. Wang request)

OTHER FUTURE WORK

- **IMU system** — needs timing data before horizon-based evaluation is meaningful
- **VLM point placement** — $\sim$1 month out, pending compute access for RL training
- **CAD model** — automated mechanical rig for aligned RGB/IR data collection
- **Real-time projection bugfixes** — remaining issues from 4-day sprint, handoff to Ty
- **Compute applications** — continue applying to higher tiers as proposals are prepared
- **Echotrail ablation** — side focus, ongoing

CONFUSED?

Questions & Discussion

jvaught@sc.edu